

GWCET

Python MCQs Assessment

100 questions | 50 code-snippet MCQs | 1 mark each

Student Name: _____

Department: _____

1. What is printed?

```
items = [1, 2, 3]
print(items)
```

- A. [1, 2, 3]
- B. (1, 2, 3)
- C. {1, 2, 3}
- D. TypeError

2. What is printed?

```
items = [1, "two", True]
print(items[1])
```

- A. two
- B. 1
- C. True
- D. IndexError

3. What is printed?

```
items = list()
print(items)
```

- A. []
- B. ()
- C. {}
- D. None

4. What is printed?

```
letters = list("Ada")
print(letters)
```

- A. ['A', 'd', 'a']
- B. ['Ada']
- C. Ada
- D. ('A', 'd', 'a')

5. What is printed?

```
numbers = list(range(4))
print(numbers)
```

- A. [0, 1, 2, 3]
- B. [1, 2, 3, 4]
- C. [0, 1, 2, 3, 4]
- D. range(4)

6. What is printed?

```
fruits = ["apple", "orange", "mango"]  
print(fruits[0])
```

- A. apple
- B. orange
- C. mango
- D. IndexError

7. What is printed?

```
fruits = ["apple", "orange", "mango"]  
print(fruits[-1])
```

- A. mango
- B. apple
- C. orange
- D. -1

8. What is printed?

```
nums = [10, 20, 30, 40, 50]  
print(nums[:3])
```

- A. [10, 20, 30]
- B. [10, 20, 30, 40]
- C. [20, 30, 40]
- D. [10, 20]

9. What is printed?

```
nums = [10, 20, 30, 40, 50]  
print(nums[1::2])
```

- A. [20, 40]
- B. [10, 30, 50]
- C. [20, 30, 40]
- D. [50, 30, 10]

10. What is printed?

```
nums = [10, 20, 30, 40]  
print(nums[::-1])
```

- A. [40, 30, 20, 10]
- B. [10, 20, 30, 40]
- C. [20, 30, 40]
- D. [40, 20]

11. What is printed?

```
nums = [1, 2, 3]  
print(nums[10:])
```

- A. []
- B. IndexError
- C. None
- D. [3]

12. What is printed?

```
fruits = ["apple", "banana", "cherry"]  
fruits[1] = "blueberry"  
print(fruits)
```

- A. ['apple', 'blueberry', 'cherry']
- B. ['blueberry', 'banana', 'cherry']
- C. ['apple', 'banana', 'blueberry']
- D. ['apple', 'banana', 'cherry', 'blueberry']

13. What is printed?

```
values = [1, 2, 3, 4]  
values[1:3] = [8]  
print(values)
```

- A. [1, 8, 4]
- B. [1, 8, 3, 4]
- C. [1, 2, 8, 4]
- D. [8, 1, 4]

14. What is printed?

```
values = [1, 2]  
values[1:2] = [7, 8]  
print(values)
```

- A. [1, 7, 8]
- B. [1, 7]
- C. [7, 8]
- D. [1, 2, 7, 8]

15. What is printed?

```
fruits = ["apple", "banana"]  
fruits.append("mango")  
print(fruits)
```

- A. ['apple', 'banana', 'mango']
- B. ['mango', 'apple', 'banana']
- C. ['apple', 'banana']
- D. ['apple', 'banana', ['mango']]

16. What is printed?

```
items = [1, 2]  
items.append([3, 4])  
print(items)
```

- A. [1, 2, [3, 4]]
- B. [1, 2, 3, 4]
- C. [[1, 2], [3, 4]]
- D. [3, 4, 1, 2]

17. What is printed?

```
fruits = ["apple", "mango"]  
fruits.insert(1, "banana")  
print(fruits)
```

- A. ['apple', 'banana', 'mango']
- B. ['banana', 'apple', 'mango']
- C. ['apple', 'mango', 'banana']
- D. ['apple', 'banana']

18. What is printed?

```
items = [1, 2]
items.extend([3, 4])
print(items)
```

- A. [1, 2, 3, 4]
- B. [1, 2, [3, 4]]
- C. [[1, 2], [3, 4]]
- D. [3, 4]

19. What is printed?

```
fruits = ["apple", "banana", "banana", "mango"]
fruits.remove("banana")
print(fruits)
```

- A. ['apple', 'banana', 'mango']
- B. ['apple', 'mango']
- C. ['apple', 'banana', 'banana', 'mango']
- D. ['apple', 'mango', 'banana']

20. What is printed?

```
items = [10, 20, 30]
removed = items.pop()
print(removed, items)
```

- A. 30 [10, 20]
- B. 10 [20, 30]
- C. 30 [10, 20, 30]
- D. 20 [10, 30]

21. What is printed?

```
items = [10, 20, 30]
removed = items.pop(0)
print(removed, items)
```

- A. 10 [20, 30]
- B. 30 [10, 20]
- C. 0 [10, 20, 30]
- D. 10 [10, 20, 30]

22. What is printed?

```
items = [1, 2, 3, 4]
del items[1:3]
print(items)
```

- A. [1, 4]
- B. [1, 2, 4]
- C. [2, 3]
- D. [1, 2, 3, 4]

23. What is printed?

```
items = [1, 2, 3]
items.clear()
print(items)
```

- A. []
- B. None
- C. [1, 2, 3]
- D. ()

24. What is printed?

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

- A. [1, 2, 3]
- B. [1, 2]
- C. [3, 1, 2]
- D. TypeError

25. What is printed?

```
a = [1, 2, 3]
b = a[:]
b[0] = 99
print(a, b)
```

- A. [1, 2, 3] [99, 2, 3]
- B. [99, 2, 3] [99, 2, 3]
- C. [1, 2, 3] [1, 2, 3]
- D. [99, 2, 3] [1, 2, 3]

26. What is printed?

```
a = [[1, 2], [3, 4]]
b = a.copy()
b[0][0] = 99
print(a)
```

- A. [[99, 2], [3, 4]]
- B. [[1, 2], [3, 4]]
- C. [[99], [3, 4]]
- D. TypeError

27. What is printed?

```
a = [1, 2, 3]
b = [4, 5]
print(a + b)
```

- A. [1, 2, 3, 4, 5]
- B. [5, 7, 3]
- C. [1, 2, 3, [4, 5]]
- D. TypeError

28. What is printed?

```
values = [1, 2]
print(values * 2)
```

- A. [1, 2, 1, 2]
- B. [2, 4]
- C. [1, 1, 2, 2]
- D. [[1, 2], [1, 2]]

29. What is printed?

```
fruits = ["apple", "mango"]
print("apple" in fruits)
```

- A. True
- B. False
- C. apple
- D. 1

30. What is printed?

```
fruits = ["apple", "mango"]  
print("grape" not in fruits)
```

- A. True
- B. False
- C. grape
- D. None

31. What is printed?

```
nums = [1, 2, 3, 8, 9]  
print(len(nums))
```

- A. 5
- B. 4
- C. 9
- D. 23

32. What is printed?

```
nums = [4, 2, 7]  
print(min(nums))
```

- A. 2
- B. 4
- C. 7
- D. 3

33. What is printed?

```
nums = [4, 2, 7]  
print(max(nums))
```

- A. 7
- B. 4
- C. 2
- D. 13

34. What is printed?

```
nums = [4, 2, 7]  
print(sum(nums))
```

- A. 13
- B. 7
- C. 3
- D. 28

35. What is printed?

```
nums = [1, 2, 3]  
nums.reverse()  
print(nums)
```

- A. [3, 2, 1]
- B. [1, 2, 3]
- C. [2, 3, 1]
- D. None

36. What is printed?

```
nums = [1, 2, 3]
print(list(reversed(nums)))
```

- A. [3, 2, 1]
- B. [1, 2, 3]
- C. reversed([1, 2, 3])
- D. (3, 2, 1)

37. What is printed?

```
nums = [3, 1, 2]
print(sorted(nums))
```

- A. [1, 2, 3]
- B. [3, 1, 2]
- C. None
- D. [3, 2, 1]

38. What is printed?

```
nums = [3, 1, 2]
result = nums.sort()
print(nums, result)
```

- A. [1, 2, 3] None
- B. [3, 1, 2] [1, 2, 3]
- C. [1, 2, 3] [1, 2, 3]
- D. None [1, 2, 3]

39. What is printed?

```
nums = [-10, 5, -3, 2]
nums.sort(key=abs, reverse=True)
print(nums)
```

- A. [-10, 5, -3, 2]
- B. [2, -3, 5, -10]
- C. [-10, -3, 2, 5]
- D. [5, 2, -3, -10]

40. What is printed?

```
names = ["Alice", "alice", "Charlie"]
names.sort(key=str.lower)
print(names)
```

- A. ['Alice', 'alice', 'Charlie']
- B. ['Alice', 'Charlie', 'alice']
- C. ['alice', 'Alice']
- D. ['Charlie', 'Alice', 'alice']

41. What is printed?

```
fruits = ["apple", "banana", "apple"]
print(fruits.count("apple"))
```

- A. 2
- B. 1
- C. 0
- D. 3

42. What is printed?

```
fruits = ["apple", "banana", "apple"]  
print(fruits.index("banana"))
```

- A. 1
- B. 0
- C. 2
- D. ValueError

43. What is printed?

```
fruits = ["apple", "banana", "apple"]  
print(fruits.index("apple", 1))
```

- A. 2
- B. 0
- C. 1
- D. The value is absent

44. What is printed?

```
fruits = ["apple", "mango"]  
print(list(enumerate(fruits)))
```

- A. [(0, 'apple'), (1, 'mango')]
- B. [('apple', 0), ('mango', 1)]
- C. ['apple', 'mango']
- D. [0, 1]

45. What is printed?

```
fruits = ["apple", "mango"]  
print(list(reversed(fruits)))
```

- A. ['mango', 'apple']
- B. ['apple', 'mango']
- C. ('mango', 'apple')
- D. ['mango']

46. What is printed?

```
fruits = ["apple", "mango"]  
for i, fruit in enumerate(fruits):  
    print(i, fruit)
```

- A. 0 apple then 1 mango
- B. apple 0 then mango 1
- C. 1 apple then 2 mango
- D. mango then apple

47. What is printed?

```
items = ["a", "b", "c"]  
for item in reversed(items):  
    print(item)
```

- A. c then b then a
- B. a then b then c
- C. 2 then 1 then 0
- D. ['c', 'b', 'a']

48. What is printed?

```
items = [1, 2, 3]
for index in range(len(items)):
    print(index, items[index])
```

- A. 0 1 then 1 2 then 2 3
- B. 1 0 then 2 1 then 3 2
- C. 1 then 2 then 3
- D. 0 then 1 then 2

49. What is printed?

```
values = ["a", "b"]
print(values[5])
```

- A. IndexError
- B. None
- C. b
- D. []

50. What is printed?

```
values = list((1, 2))
print(values)
```

- A. [1, 2]
- B. (1, 2)
- C. {1, 2}
- D. TypeError

51. What does del values[1] do?

- A. Deletes the item at index 1
- B. Removes the value 1
- C. Returns the item at index 1
- D. Clears the whole list

52. What does del values[1:3] do?

- A. Deletes the selected slice
- B. Deletes only index 3
- C. Returns a copied slice
- D. Sorts the selected slice

53. What remains after values.clear()?

- A. []
- B. None
- C. The final value
- D. A tuple

54. What does b = a create when a is a list?

- A. An alias to the same list object
- B. An independent shallow copy
- C. An independent deep copy
- D. A tuple view

55. What happens after `a = [1]`; `b = a`; `b.append(2)`?

- A. Both a and b are [1, 2]
- B. Only b is [1, 2]
- C. Only a is [1, 2]
- D. Both lists are unchanged

56. Which expression makes a shallow copy of a list?

- A. `a[:]`
- B. `b = a`
- C. `id(a)`
- D. `a is b`

57. Which method makes a shallow copy of a list?

- A. `a.copy()`
- B. `a.alias()`
- C. `a.cloneDeep()`
- D. `a.freeze()`

58. For a flat list, what happens after `b = a[:]`; `b[0] = 99`?

- A. a remains unchanged
- B. a changes at index 0
- C. Both names become invalid
- D. a becomes a tuple

59. What do a shallow copy and original share for nested lists?

- A. Their inner list objects
- B. Their outer list object
- C. No objects at all
- D. Only their lengths

60. Which function creates independent copies of nested lists?

- A. `copy.deepcopy`
- B. `list.copy`
- C. `list.extend`
- D. `reversed`

61. Why does `d[0][0] = 99` affect c after `d = c[:]`?

- A. The inner list is shared by the shallow copy
- B. The outer list is immutable
- C. Indexing always deep-copies values
- D. Slicing reverses nested lists

62. What does `a + b` do for two lists?

- A. Creates a concatenated list
- B. Performs element-wise numeric addition
- C. Mutates only a
- D. Creates a set

63. What does `[1, 2] * 2` produce?

- A. `[1, 2, 1, 2]`
- B. `[2, 4]`
- C. `[1, 2, 2, 1]`
- D. A `TypeError`

64. Which expression is valid list concatenation?

- A. `a + b + [6]`
- B. `a + b + 6`
- C. `a + 6`
- D. `6 + a`

65. What does `'apple'` in `fruits` test?

- A. Whether the value appears in the list
- B. Whether `apple` is a valid index
- C. Whether the list is sorted
- D. Whether fruit values are unique

66. What does `'grape'` not in `fruits` test?

- A. Whether `grape` is absent from the list
- B. Whether `grape` is the final item
- C. Whether the list is empty
- D. Whether `grape` is immutable

67. What is the typical complexity of list membership testing?

- A. $O(n)$
- B. $O(1)$
- C. $O(\log n)$
- D. $O(n^2)$

68. Why can list membership take longer as a list grows?

- A. It may search items linearly
- B. It sorts the list before every check
- C. It removes duplicates first
- D. It converts the list into a set

69. What does `len([4, 5, 6])` return?

- A. 3
- B. 2
- C. 6
- D. 0

70. What does `min([4, 2, 7])` return?

- A. 2
- B. 4
- C. 7
- D. A `ValueError`

71. What does `max([4, 2, 7])` return?

- A. 7
- B. 4
- C. 2
- D. The list length, 3

72. What does `sum([4, 2, 7])` return?

- A. 13
- B. The largest value, 7
- C. The list length, 3
- D. The product, 56

73. How are strings compared by `min` and `max`?

- A. Lexicographically
- B. By string length only
- C. By insertion position only
- D. As numeric values always

74. What does `max(['Alice', 'Bob', 'Charlie'])` compare to choose a result?

- A. Lexicographic character order
- B. The shortest name
- C. The earliest inserted name
- D. The name with most vowels

75. Which built-in reports the number of elements in a list?

- A. `len`
- B. `sum`
- C. `max`
- D. `ord`

76. What does `values.reverse()` do?

- A. Reverses the same list in place
- B. Returns a sorted new list
- C. Creates a tuple in reverse order
- D. Removes duplicate values

77. What does `list(reversed(values))` produce?

- A. A new list in reverse order
- B. The same list reversed in place
- C. A sorted list
- D. A list of index values

78. What does `sorted(values)` return?

- A. A new sorted list
- B. None after sorting in place
- C. A reversed iterator
- D. A set of unique items

79. What does `values.sort()` do?

- A. Sorts the existing list in place
- B. Returns a new sorted list
- C. Reverses the existing list
- D. Raises an error for numbers

80. What does `values.sort()` return?

- A. None
- B. The sorted list
- C. A sort iterator
- D. True

81. What does `sorted(words, key=len, reverse=True)` prioritize?

- A. Longest strings first
- B. Shortest strings first
- C. Alphabetical order only
- D. Original index order only

82. What does `values.sort(key=abs, reverse=True)` use for ordering?

- A. Descending absolute values
- B. Ascending raw values
- C. Descending string values
- D. Original insertion positions

83. Which key can sort names without case sensitivity?

- A. `str.lower`
- B. `str.uppercase`
- C. `len.lower`
- D. `casefold.index`

84. Which operation keeps the original list unchanged?

- A. `sorted(values)`
- B. `values.sort()`
- C. `values.reverse()`
- D. `values.clear()`

85. What does `fruits.count('apple')` return?

- A. The number of 'apple' occurrences
- B. The first 'apple' index
- C. The final 'apple' index
- D. A list without apples

86. What does `fruits.index('apple')` return?

- A. The first matching index
- B. The number of matches
- C. The final matching index
- D. A boolean result

87. What happens when `list.index(value)` cannot find value?

- A. It raises `ValueError`
- B. It raises `IndexError`
- C. It returns `-1`
- D. It returns `None`

88. What does `fruits.index('apple', 1)` control?

- A. The index where the search begins
- B. The count of apples to remove
- C. The list position to insert apple
- D. The length of the result

89. What does `['a', 'b', 'a'].count('a')` return?

- A. 2
- B. 1
- C. 3
- D. A `ValueError`

90. What does `['a', 'b', 'a'].index('a')` return?

- A. 0
- B. 1
- C. 2
- D. The value `'a'`

91. Which method is useful for checking how many times one value appears?

- A. `count`
- B. `index`
- C. `reverse`
- D. `clear`

92. Which loop iterates directly over list values?

- A. `for fruit in fruits`
- B. `for fruit at fruits`
- C. `for fruits in fruit`
- D. `loop fruit from fruits`

93. Which loop provides both an index and a value?

- A. `for i, fruit in enumerate(fruits)`
- B. `for i in fruits.index()`
- C. `for fruit in range(fruits)`
- D. `for i, fruit in reversed(enumerate)`

94. What does `enumerate(fruits)` provide during iteration?

- A. Index-value pairs
- B. Only index values
- C. Only list lengths
- D. Sorted values

95. Which loop accesses items by explicit numeric index?

- A. `for i in range(len(fruits))`
- B. `for i in fruits.values()`
- C. `for i in enumerate(len(fruits))`
- D. `for i in list(fruits, index)`

96. What does `reversed(fruits)` provide in a for loop?

- A. Items from last to first
- B. Items in sorted order
- C. Only even-index items
- D. A shallow copy

97. Which pattern is preferred when both position and value are needed?

- A. `enumerate(fruits)`
- B. `range(fruits)`
- C. `len(enumerate)`
- D. `fruits.index()`

98. What does this print: `for item in ['x', 'y']: print(item)`?

- A. x then y
- B. 0 then 1
- C. y then x
- D. The list length only

99. What is the first index produced by `enumerate(['a', 'b'])` by default?

- A. 0
- B. 1
- C. -1
- D. 2

100. What is the first value produced by `reversed([1, 2, 3])`?

- A. 3
- B. 1
- C. 2
- D. The final index, 2